# API Security Risk Analysis

## Future Interns – Cyber Security Task 3 (2026)

## API Tested: JSONPlaceholder - https://jsonplaceholder.typicode.com

# TABLE OF CONTENTS

# Executive Summary

This report explains a read only API Security Risk Analysis performed on the public JSONPlaceholder API. The objective was to identify common API security risk, assess authentication, data exposure, recommend remediation, response headers, exposed data and security posture. This assessment was performed using Postman and followed ethical testing practices with no attempts made to modify data.

Application Programming Interfaces (APIs) are important components of modern software systems that enables communication between web applications, mobile applications and cloud services. As organizations increasingly rely on APIs to exchange data and deliver services, securing these interfaces has become a critical aspect of cybersecurity.

The analysis identified several common API security risks that includes open endpoints that do not require authentication, the apparent absence of rate limiting and a lack of authorization checks to restrict access to user data. Each identified risk was assessed according to its potential impact on confidentiality, integrity and availability.

Practical remediation recommendations are provided based on API security best practices and the OWASP API Security Top 10. These recommendations include implementing strong authentication mechanisms. Limiting the amount of data returned in API response by applying rate limiting to prevent abuse and ensuring all communications are protected using HTTPS.

# Methodology

**Endpoints Tested**: /, /posts, /users, /users/1

**Request**: GET

**Tools**: Postman for API requests and header inspection, Browser DevTools for response validation

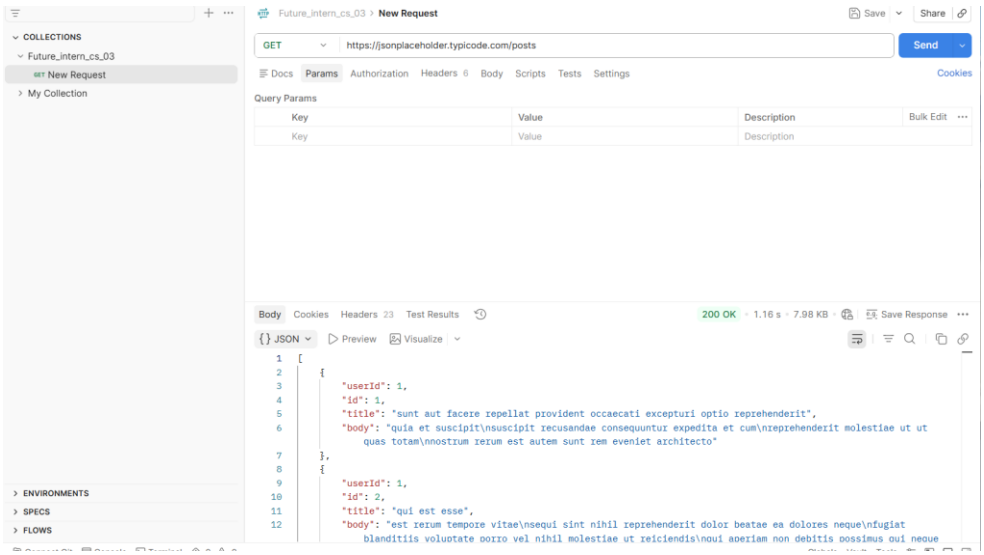
**Ethical scope**: Only public demo endpoints were analysed such as: No exploitation, denial-of-service, private API testing

# Evidence findings:

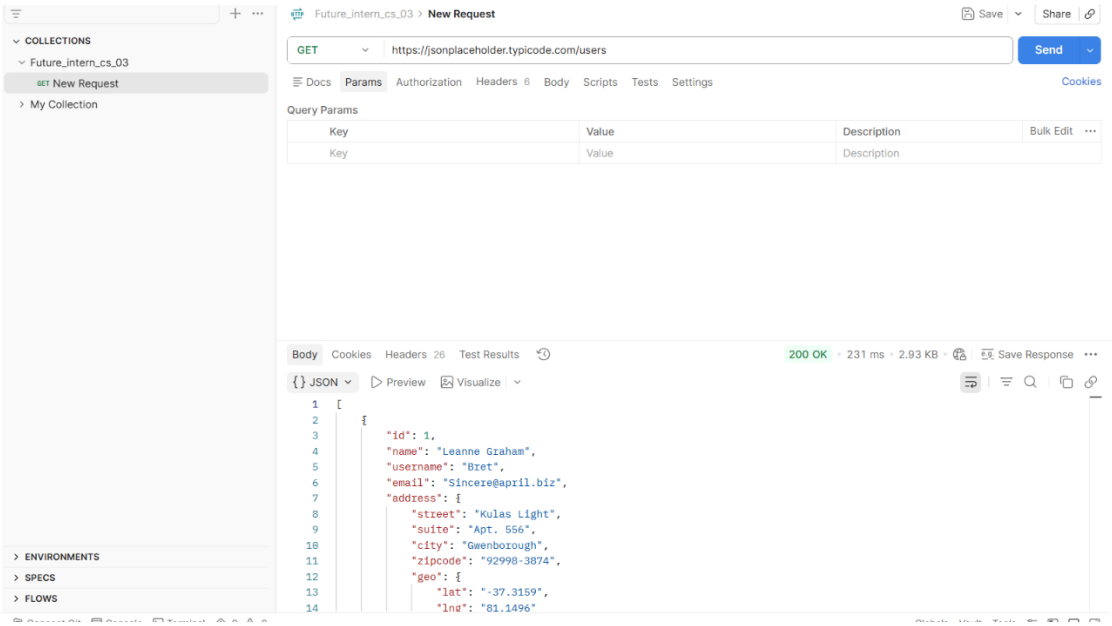
1. **/posts Endpoint**

Displays a JSON array of posts.

- It demonstrates that data is publicly available without authentication
- It confirms proper JSON formatting and 200 Ok status.
- The evidence of open endpoint risks anyone can retrieve all posts.



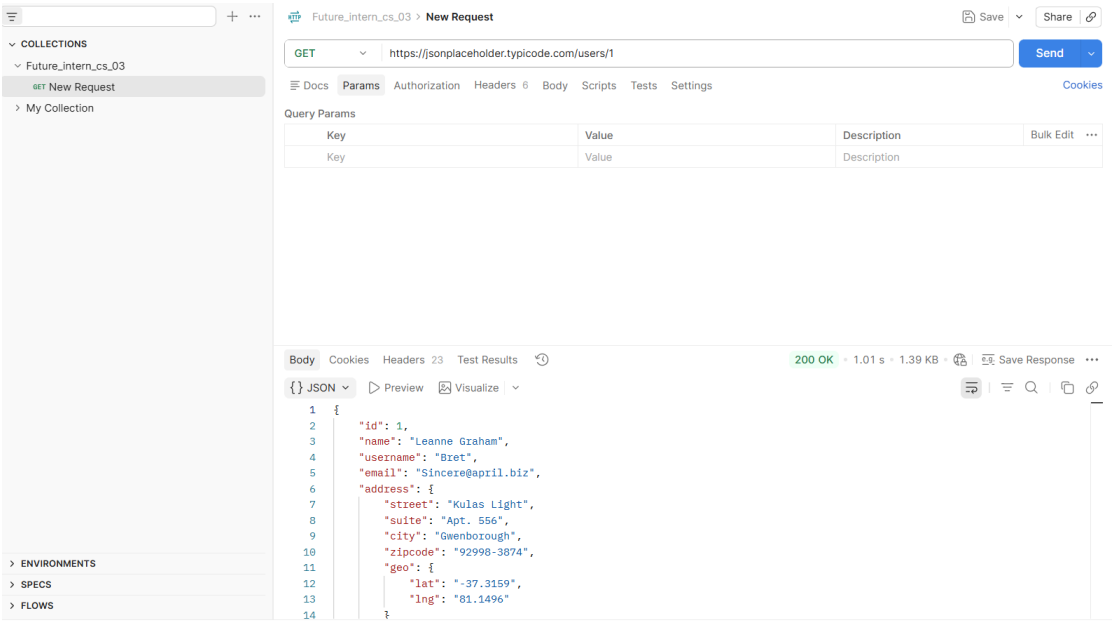

2. **/users Endpoint**
   It shows a JSON array of user objects.
   - It includes sensitive fields such as email, address and geo coordinates.
   - The evidence of excessive data exposure it has more information than necessary is returned.
   - It highlights privacy concerns if this were a production API.




### 3. /users/1 Endpoint

Displays a single user object.

- It confirms direct object access is possible without authentication.
- The evidence of broken access control risks it shows that attackers could enumerate user IDs.
- It shows nested data structure that increase exposure.

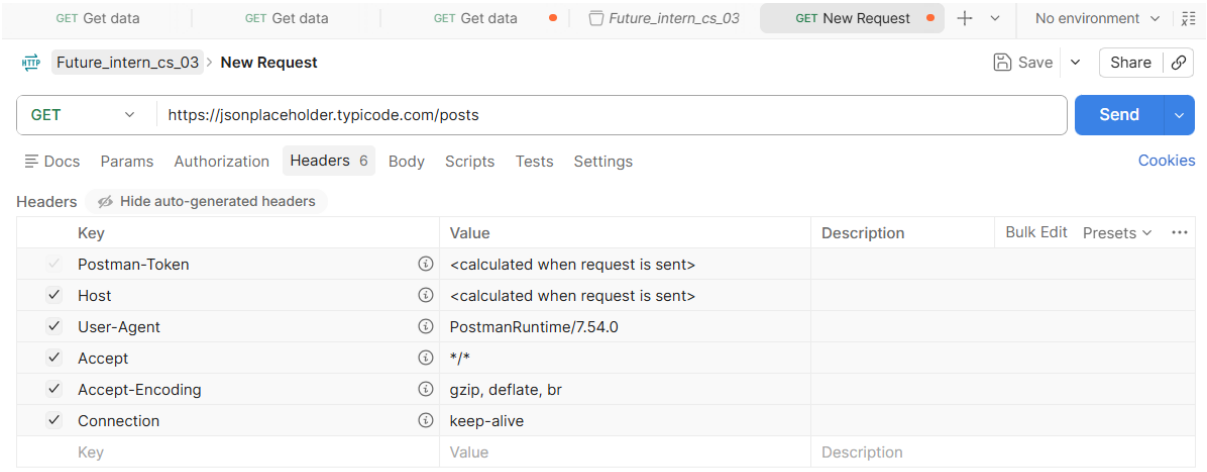



### 4. Header for /posts

Shows response headers inspected via Postman/DEVTools

- It confirms content-type: application/json.

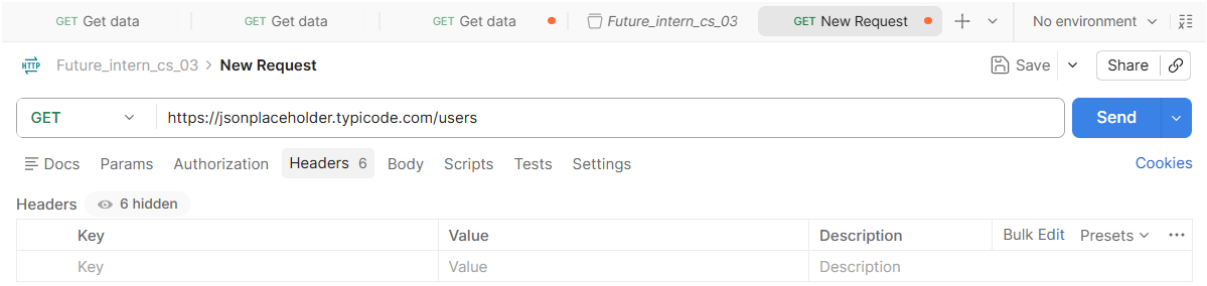
- It shows missing security headers.
- It shows evidence of weak security controls

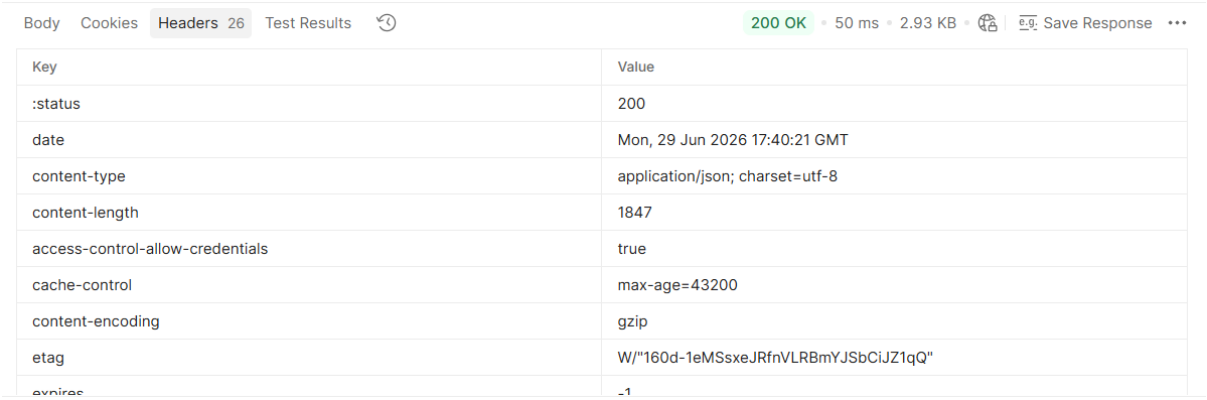



### 5. Headers for /users

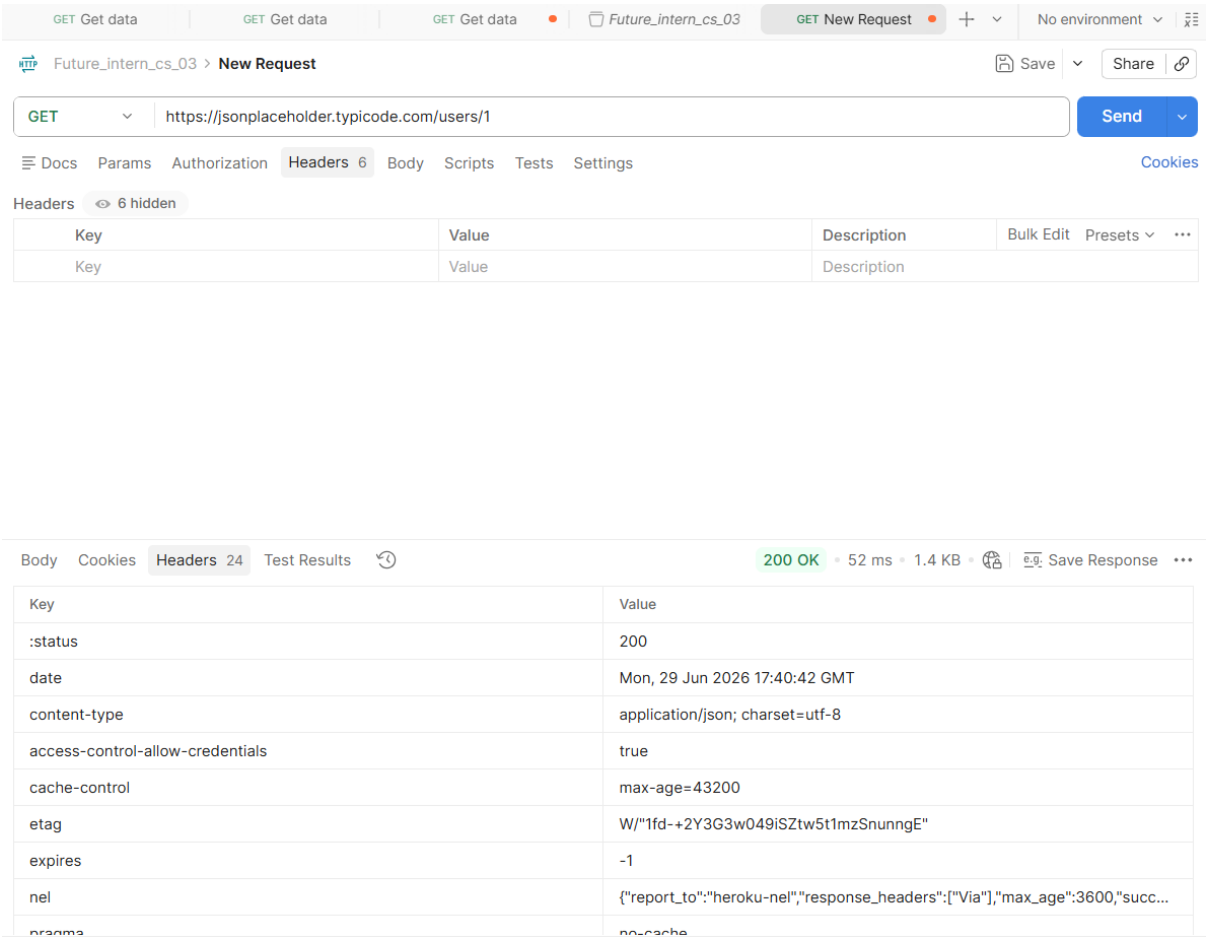
Shows response headers for user list.

- It shows the same missing headers as posts
- It confirms no authentication or authorization tokens required
- It shows evidence of authentication or authorization tokens required
- It shows evidence of unauthenticated access risks




6. **Headers for /users/1**
   Shows response headers for single user object.
   - It confirms consistent header behaviour across endpoints
   - It reinforces lack of security headers and authentication
   - It shows evidence of systemic API weaknessess







| Key | Value |
|---|---|
| :status | 200 |
| date | Mon, 29 Jun 2026 17:40:42 GMT |
| content-type | application/json; charset=utf-8 |
| access-control-allow-credentials | true |
| cache-control | max-age=43200 |
| etag | W/"1fd-+2Y3G3w049iSZtw5t1mzSnunngE" |
| expires | -1 |
| nel | {"report_to":"heroku-nel","response_headers":["Via"],"max_age":3600,"succ... |
| pragma | no-cache |

# Findings and Analysis

| ENDPOINT | OBSERVATION | RISK TYPE | SEVERITY |
|---|---|---|---|
| /posts | No authentication required returns all posts | Open endpoint | High |
| /Users | Exposes user emails and geo coordinates | Excessive data exposure | Medium |
| /users/1 | Direct object access possible without authorization | Missing authorization | High |
| All endpoints | No visible rate limiting | Missing rate limiting | Low |

# Business Impact

- The open endpoints allow data harvesting and profiling.
- The exposed user emails and geo data could lead to privacy violation, phishing and/or location tracking.
- The missing security headers reduce protection against clickjacking and content injection.
- Loss of customer trust.
- Damage to the organization’s reputation.
- Data misuse by malicious actors.
  Overall, these risks while moderate in a demo API would be critical in a production SaaS environment.

If these weaknesses existed in a production SaaS application attackers could collect customer information, abuse API resources and violate privacy regulations. Such incidents could result in financial losses, penalty and reduced customer confidence.

# Remediation Recommendations

- Add rate limiting to prevent abuse
- Apply role-based access control for user specific data
- Limit response fields to necessary data only
- Implement API key or token-based authentication
- Restrict exposed data
- Enforce HTTPS
- Validate all API inputs before processing requests

# Conclusion

This assessment successfully shows the security posture of the JSONPlaceholder demo API using safe and ethical testing practices. I have identified several common API security risks including open endpoints, excessive data exposure, lack of authorization controls and the apparent absence of rate limiting. While these characteristics are acceptable for a public educational API, they would represent significant security concerns in a production environment. Applying strong authentication, authorization, input validation, rate limiting and secure data handling practices would significantly improve API security and reduce organizational risks.

# References

- **JSONPlaceholder (Test API)**
  https://jsonplaceholder.typicode.com
- **Postman**
  https://www.postman.com
- **OWASP API Security Top 10**